

Industrial Internship Report on "Design and Development of a Web-Based Marketplace Application"

**Prepared by
Aditya Kalme**

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time.

The project involved the development of a web-based application designed to function as an online marketplace, connecting customers and merchants in a seamless and efficient manner. The platform aims to facilitate interactions between service providers and customers, offering a cost-effective, user-friendly solution that benefits both parties.

This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1	Preface	3
2	Introduction	5
2.1	About UniConverge Technologies Pvt Ltd	5
2.2	About upskill Campus	9
2.3	Objective	11
2.4	Reference	11
2.5	Glossary	12
3	Problem Statement	13
4	Existing and Proposed solution	14
5	Proposed Design/ Model	16
5.1	High Level Diagram (if applicable)	17
5.2	Low Level Diagram (if applicable)	19
5.3	Interfaces (if applicable)	20
6	Performance Test	23
6.1	Test Plan/ Test Cases	23
6.2	Test Procedure	24
6.3	Performance Outcome	24
7	My learnings	28
8	Future work scope	30

1 Preface

The six-week internship period provided a comprehensive, hands-on experience in full-stack web development using modern frameworks and tools. The internship began with foundational learning and gradually transitioned into the design and development of a complete service marketplace web application.

During **Week 1**, I focused on understanding the fundamentals of Java programming and the core principles of Spring Boot. This included learning about object-oriented programming concepts, annotations, dependency injection, and the basics of creating RESTful APIs using Spring Boot.

In **Week 2**, I explored database connectivity and authentication mechanisms. The integration of MySQL with Spring Boot was practiced using Spring Data JPA for ORM mapping, along with implementation of middleware for request handling and authentication using JWT (JSON Web Tokens).

Week 3 was dedicated to mastering routing techniques and understanding the dependencies and principles of Spring Boot. Emphasis was placed on backend routing, modular architecture, and dependency management using Maven. I also learned about RESTful services, controller mapping, and configuration of different application modules.

In **Week 4**, the focus shifted to frontend development using the Angular.js framework. I learned about component-based architecture, two-way data binding, and routing in Angular. This week helped bridge the connection between frontend and backend by consuming REST APIs built earlier.

During **Weeks 5 and 6**, I applied the skills gained throughout the training to design and develop the final project — a multi-tenant web marketplace platform. The project involved creating user roles for merchants and customers, integrating a payment gateway, implementing secure authentication, and developing merchant dashboards and service management interfaces. The system was tested for functionality, scalability, and performance to ensure a smooth user experience.

Overall, the internship provided a structured pathway from learning individual technologies to applying them in a cohesive real-world project.

This internship offered me the opportunity to work with technologies currently in demand in the software industry, including Java Spring Boot, Angular.js, REST APIs, and relational databases. Through guided mentorship and practical tasks, I was able to understand not only how to build applications but also how to structure and manage them effectively in a professional environment.

Participating in this internship has enhanced my technical skills, problem-solving capabilities, and teamwork abilities — all of which are essential attributes for a successful career in software engineering. It provided me with the confidence to take on full-stack development projects and prepared me for future roles in the IT industry.

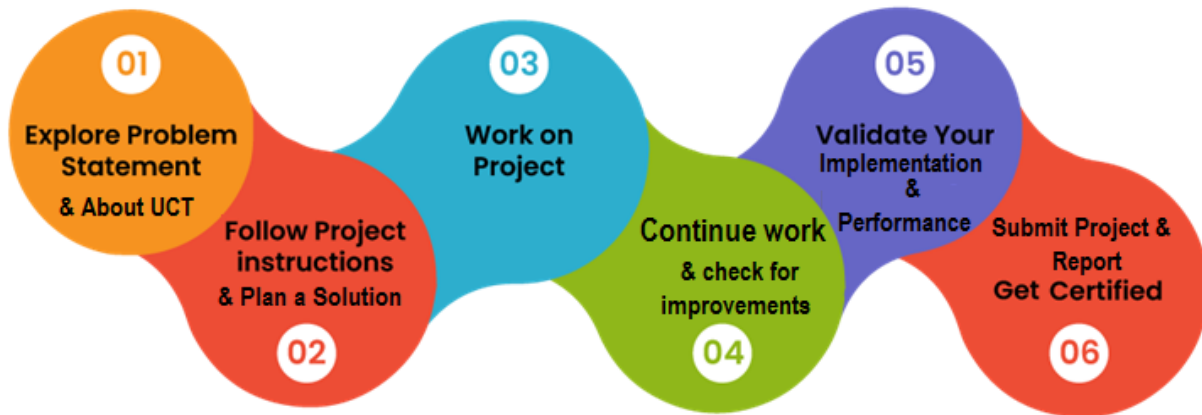


Figure 1: Planning of the Project Completion

I reflect on the successful completion of the **Dual Marketplace Application** by considering not just the final result but the journey and the growth that defined this experience. It has been a period of significant learning and I am grateful for the opportunity to have contributed to this endeavor.

Thank to my **HR manager** for their leadership and the unwavering faith instilled in me which became the catalysts for success. Thank you for fostering an environment where I felt empowered to take ownership and excel. Thank you to **USC, UCT** for providing the platform to learn, grow and achieve, for which I am proud of what we have achieved together.

To my **peers and colleagues**, both near and far, thank you. Whether through direct collaboration or the indirect support you provided during busy seasons, your insights and encouragement made a meaningful difference. I have learned a great deal from your unique perspectives, and I am honored to work alongside such a dedicated group of professionals.

To my **juniors**, I hope this experience serves as a reminder that your potential is often greater than you realize. Lean into the challenges you face, ask questions, and never underestimate the impact of your contributions. To my **peers**, let us continue to challenge one another, stay curious, and maintain the high standard of excellence that defines our team.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies** e.g. **Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



i. UCT IoT Platform ()

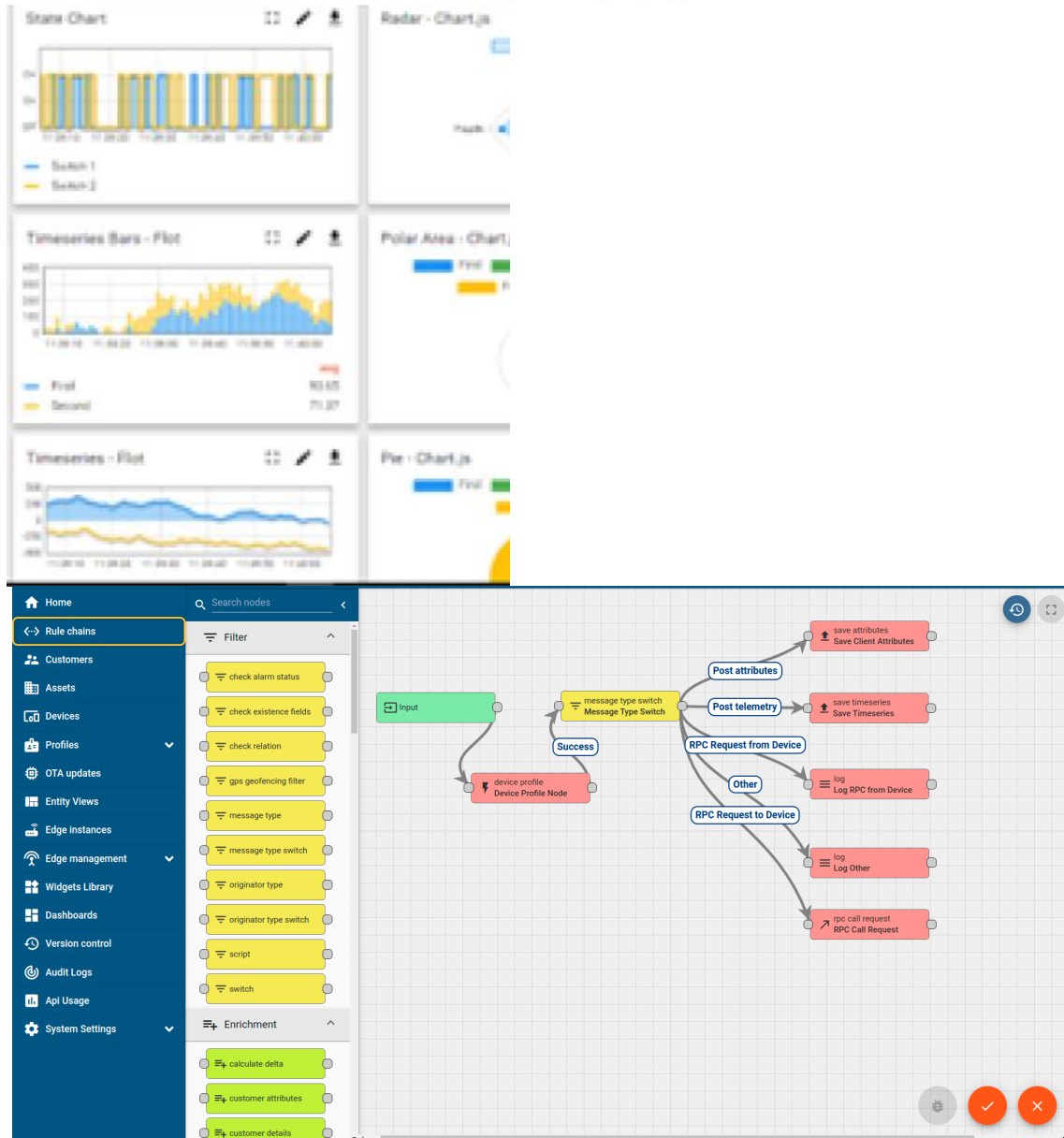
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA

- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



FACTORY WATCH

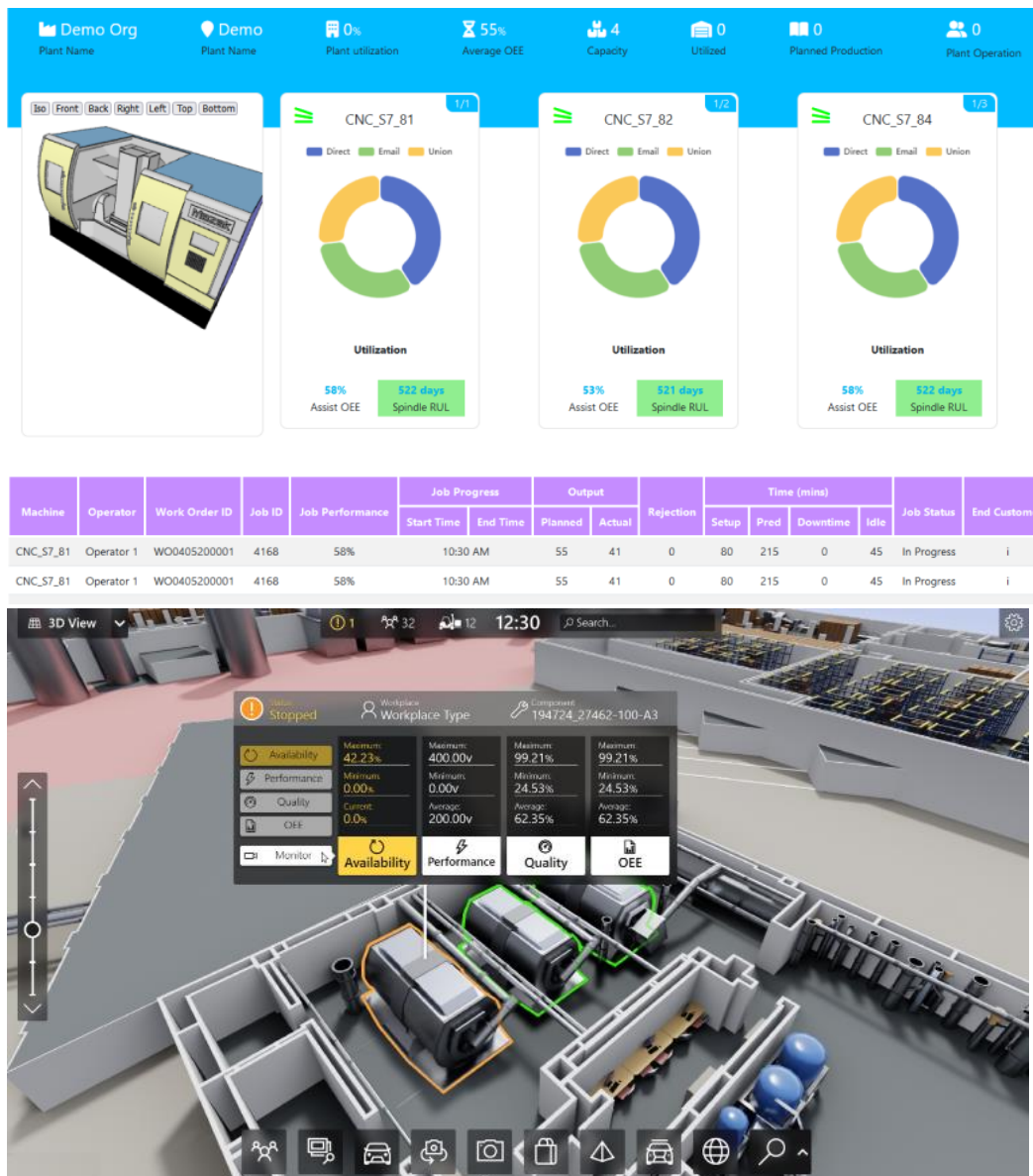
ii. Smart Factory Platform ()

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleash the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they want to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



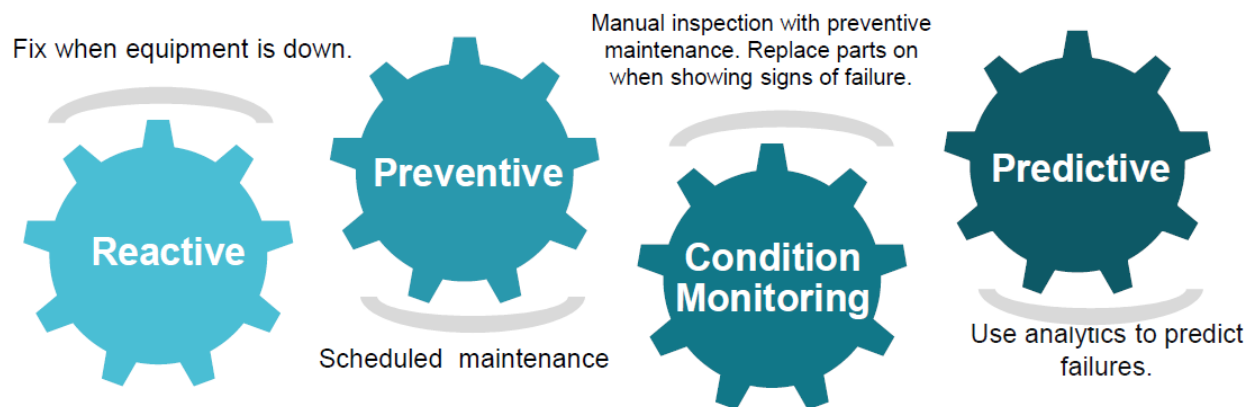


iii. based Solution

UCT is one of the early adopters of LoRAWAN technology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

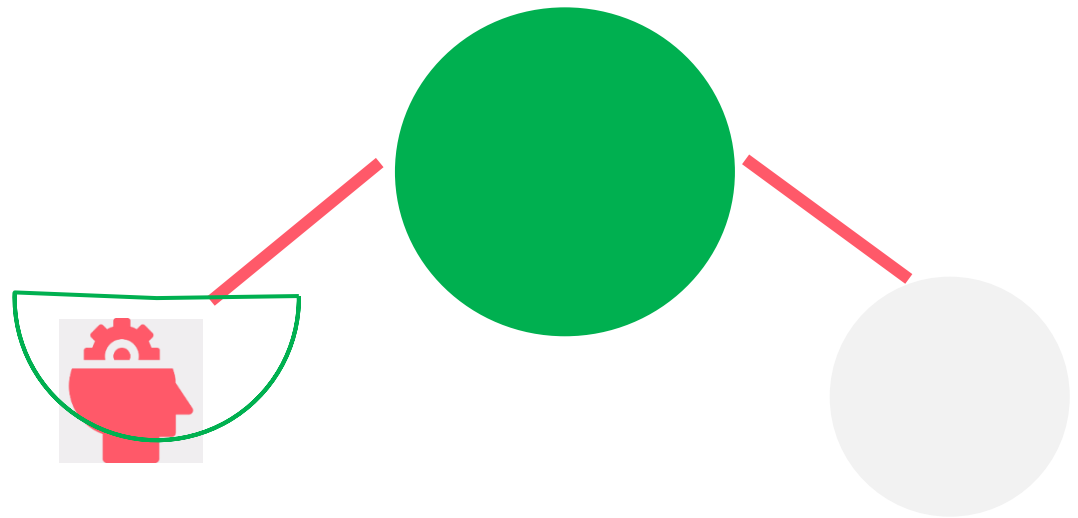
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

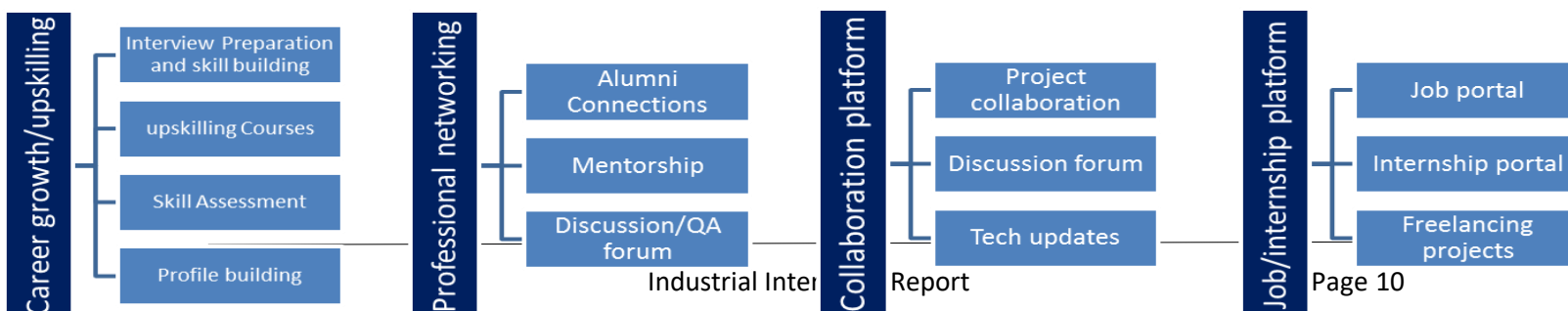
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

<https://www.upskillcampus.com/>



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- get practical experience of working in the industry.
- to solve real world problems.
- to have improved job prospects.
- to have Improved understanding of our field and its applications.
- to have Personal growth like better communication and problem solving.

2.5 Reference

- [1] [Thymeleaf – Website](#)
- [2] [Spring Boot – Documentation](#)
- [3] [Spring Boot - Guides](#)

2.6 Glossary

Acronym	Terms
API	Application Programming Interface
DI	Dependency Injection
IoC	Inversion of Control
MVC	Model-View-Controller
POJO	Plain Old Java Object
CRUD	Create, Read, Update, Delete
DAO	Data Access Object
JPA	Java Persistence API
HTML	HyperText Markup Language
SaaS	Software as a Service

3 Problem Statement

The problem statement for this text centers on the need for a **multi-tenant service-based marketplace platform** that bridges the gap between service providers (merchants) and their clients (end-users).

Essentially, the platform must solve the fragmentation of service discovery, booking, and payment processing for various local service industries (e.g., home, beauty, pet care).

Objective

The goal is to create a **frictionless, scalable B2B2C (Business-to-Business-to-Consumer) ecosystem**. By providing merchants with a "storefront-in-a-box" and customers with a "one-stop-shop," the platform solves the problem of high-friction service discovery and inefficient manual booking/payment processes.

4 Existing and Proposed solution

Existing Solutions

There are multiple existing solutions to creating **Design and Development of a Web-Based Marketplace Application** such as **Shopify**, **WooCommerce**, or bespoke **SaaS** solutions. Common limitations for them include:

- **Rigid Infrastructure:** Many off-the-shelf platforms are built primarily for physical goods. When businesses try to bolt on "service-based" features (like appointment scheduling), they often encounter high integration costs, performance degradation, or an clunky user experience.
- **Scalability Issues:** In multi-tenant environments, a single high-traffic merchant can inadvertently degrade the performance of the entire platform if resource isolation isn't strictly managed.
- **Technical Debt and Customization Barriers:** Platforms often restrict access to the underlying code or database schema to ensure "ease of use." This makes it extremely difficult for businesses to implement niche features specific to their industry without hitting a "customization wall" or incurring massive technical debt.
- **High Costs and Vendor Lock-in:** Proprietary platforms often impose hefty subscription or transaction fees, tying your business growth to their pricing models

Proposed Solution

Proposed solution is a **custom-built, multi-tenant Spring Boot web application** that integrates both **goods (inventory)** and **services (appointments)** into a single, unified database schema. By using a unified schema (Merchants, Services, Goods, Customers, Appointments, Transactions), you are creating a **cohesive marketplace** where the logic for a "product" is abstracted to support both tangible goods and time-bound services. Utilizing Spring Boot and Thymeleaf allows for a robust, secure, and highly scalable server-side rendered application. This eliminates the need for expensive third-party plugins, as you maintain full control over the codebase and data integrity. We are addressing the fragmentation problem by multi-role dashboards for merchant and customer

Value Addition

- **Unified Hybrid Marketplace:** Most platforms force a distinction between "eCommerce" (goods) and "Booking" (services). Your application adds value by treating both as a "Product," allowing a merchant to sell a physical item (e.g., a haircut kit) alongside a service (e.g., a haircut appointment) in a single transaction flow.
- **Integrated Loyalty & Wallet System:** Your schema includes `wallet_balance` for both Customers and Merchants. Expanding this into an **integrated platform credit system** (e.g., auto-refunding to the wallet for cancelled appointments) provides a much smoother user experience than external payment gateway redirects.
- **Transparent Admin Oversight:** By including a dedicated Admin Dashboard to track "queries/disputes" and their statuses, you are adding a layer of **trust and governance** that many smaller, plug-and-play marketplaces lack. This is a critical selling point for B2B2C platforms.
- **Seamless Data Continuity:** By having a shared schema, you can easily provide customers with an **integrated order history** that mixes physical goods delivery and service appointment logs, creating a complete "Consumer 360" view of their activity on your platform.

4.1 Code submission (Github link) : <https://github.com/almighty-beanos/upskillcampus/tree/main/MultiClientWebsite.java>

4.2 Report submission (Github link) : https://github.com/almighty-beanos/upskillCampus/tree/main/MultiClientWebsite_Aditya_USC_UCT.pdf
https://github.com/almighty-beanos/upskillCampus/tree/main/MultiClientWebsite_Aditya_USC_UCT.docx

5 Proposed Design/ Model

This e-commerce understands the application as a pipeline dividing the design flow into stages which diversification of work across domains. This causes any successful interaction to follow a predictable flow from the user's intent to the system outcome.

Here is the design flow pattern for your Spring Boot application, mapped to a universal lifecycle.

5.1.1 The Entry Point (Request Handling)

This is where the user initiates an action.

- **Design Principle: Validation.** Never trust incoming data.
- **Implementation:** Using Spring `@RequestMapping` or `@GetMapping`/`@PostMapping`.
- **The Flow:** The client sends an HTTP request. The `Controller` layer acts as the gatekeeper, checking if the user is authenticated and authorized (via Spring Security).

5.1.2 2. The Intermediate Stage (Orchestration)

This is the "Brain" of the application.

- **Design Principle: Decoupling.** The `Controller` should only handle "what" happens, while the `Service` handles "how" it happens.
- **The Flow:**
 - **Business Logic:** Does the customer have enough `wallet_balance`? Is the merchant's service available?
 - **Data Transformation:** Converting the data from the database (`Entity`) into a clean format for the UI (`DTO` - Data Transfer Object).
 - **Security Check:** Ensuring the merchant can only edit their *own* products.

5.1.3 3. The Persistence Stage (Data Integrity)

The point of no return.

- **Design Principle: ACID.** (Atomicity, Consistency, Isolation, Durability).
- **The Flow:** Using `@Transactional` annotations. If a transaction consists of two parts (e.g., debiting the customer and crediting the merchant), the system must ensure that if one fails, both are rolled back. This prevents "phantom money" or orphaned orders.

5.1.4 The Outcome Stage (Response & Experience)

The closure of the user's journey.

- **Design Principle: Feedback Loops.** Always let the user know the status of their action.
- **The Flow:**
 - **Success:** Redirecting to a "Thank You" page with an order confirmation ID.
 - **Failure:** Providing descriptive error messages (e.g., "Insufficient funds" instead of a generic "Server Error").
 - **Asynchronous Cleanup:** Sending emails or notifications in the background so the user doesn't have to wait for the server to finish the process.

5.2 High Level Diagram (if applicable)

(Architecture
Flow and Design
Methodology is present
below)

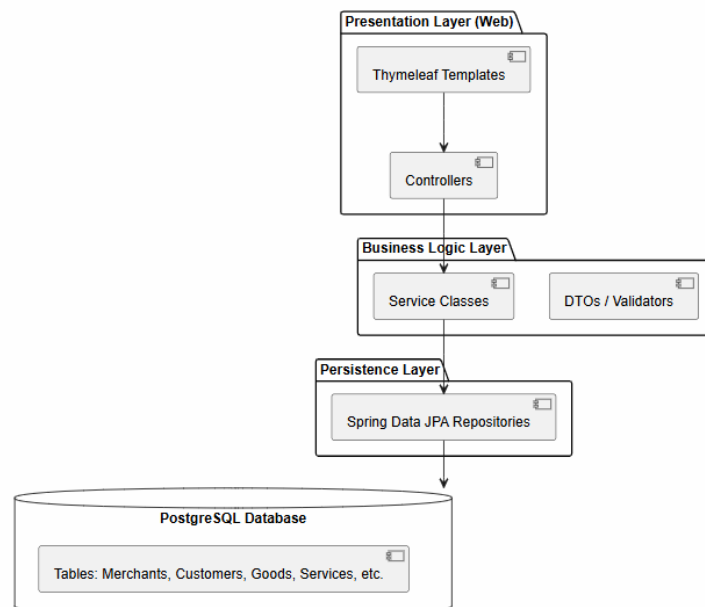


Figure 1: Architecture Flow

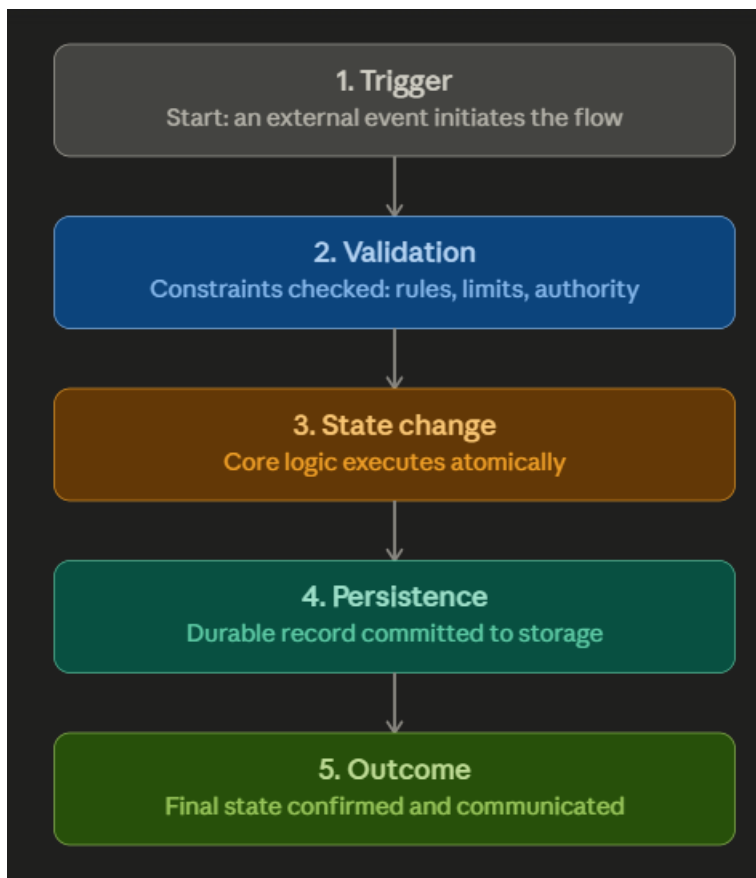


Figure 2: DESIGN METHODOLOGY

5.3 Low Level Diagram (if applicable)

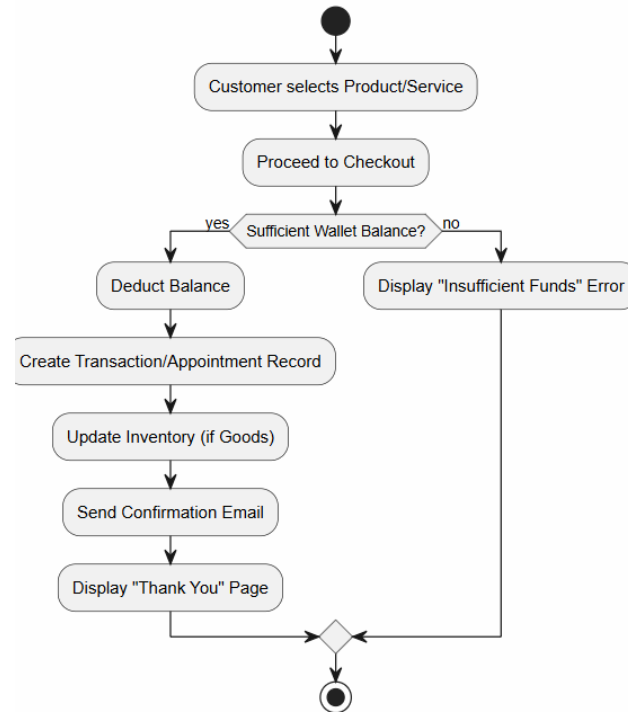


Figure 3: Checkout Process (Payment Repository)

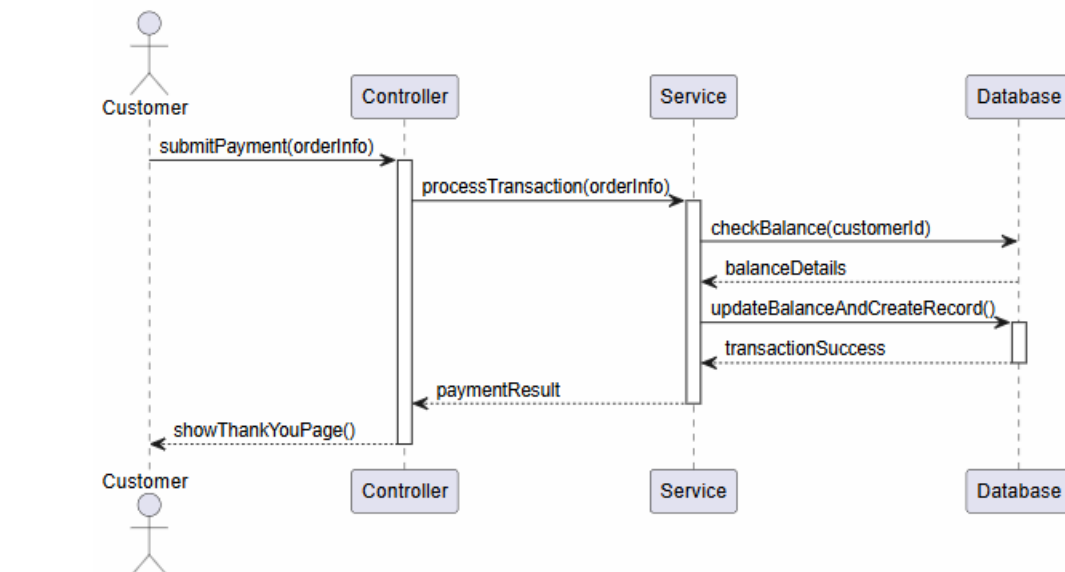


Figure 4: Payment Transaction

5.4 Interfaces (if applicable)

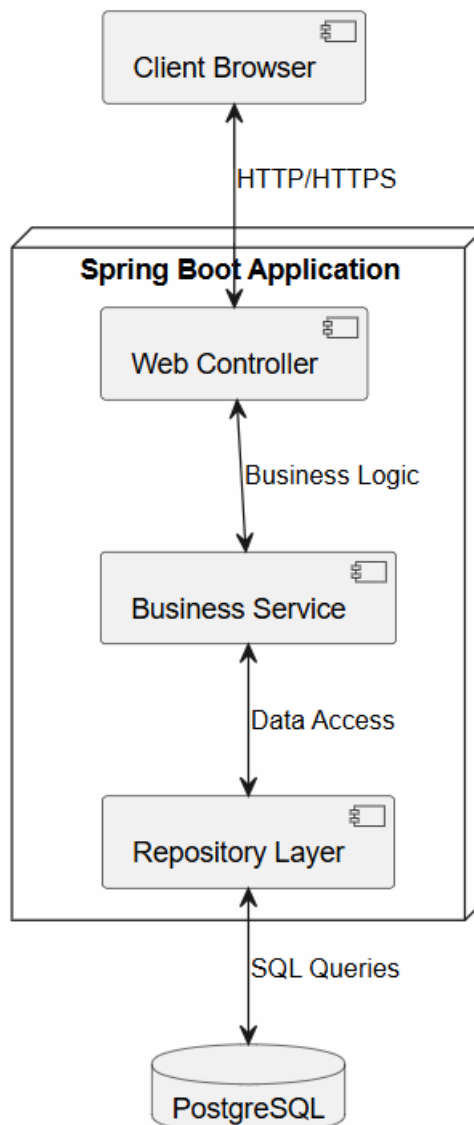


Figure 5: Block Diagram (System Overview)

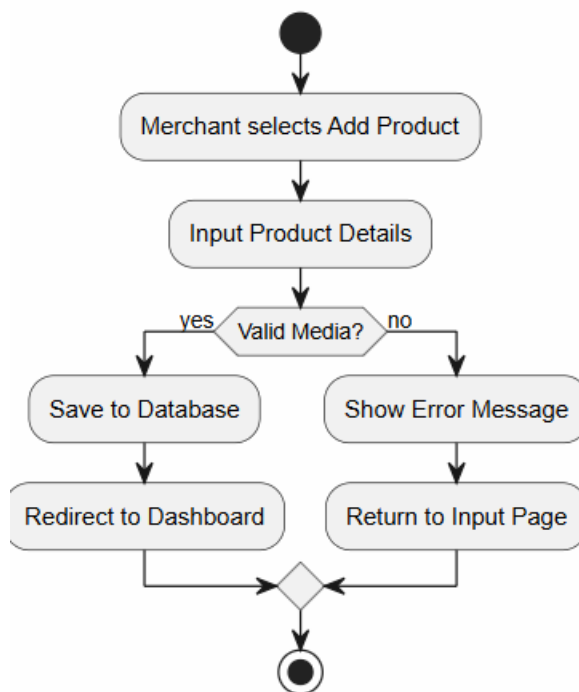


Figure 6: Inventory Decision Logic

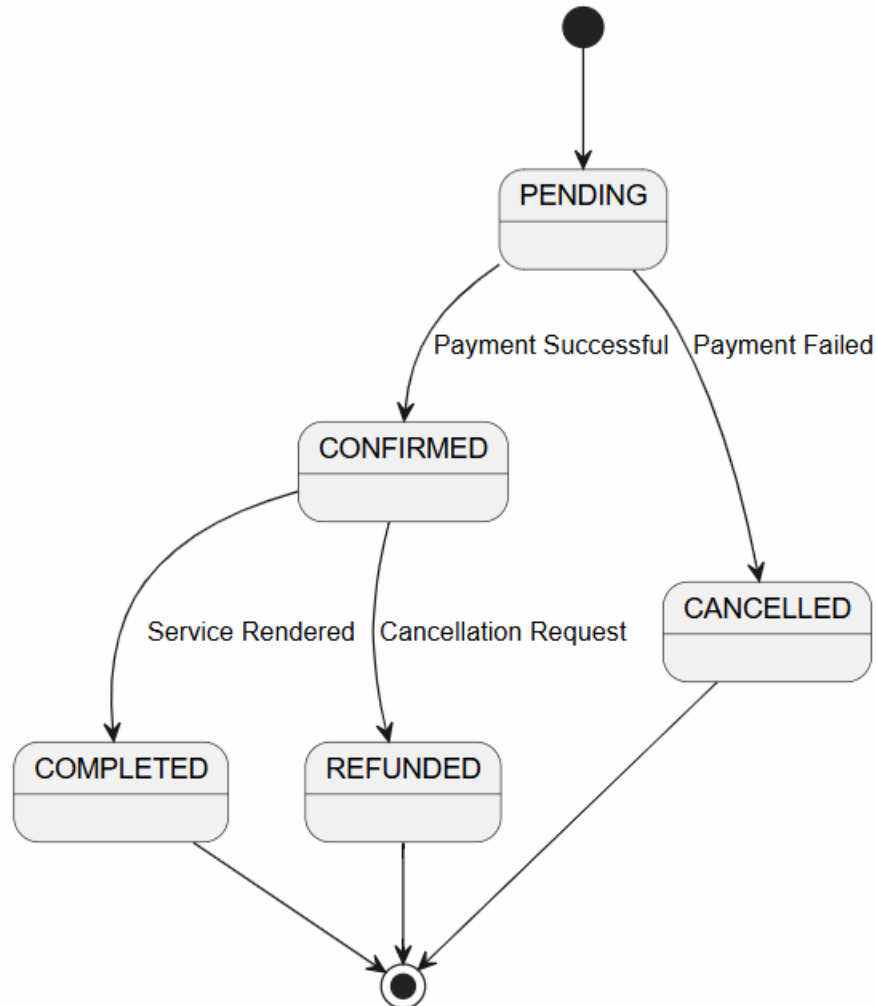


Figure 7: State Machine

6 Performance Test

Constraint	Design Handling
No access to payment gateway	In-house wallet ledger; <code>CheckoutService.transferWallet()</code> debits/credits both parties in one @Transactional method
Third-party dependency (Supabase Storage)	SupabaseStorageService makes a synchronous blocking HTTP call inside the request thread; no retry, no timeout override, no circuit breaker
Discover Page Stability	Filtering/sorting happens client-side — the entire goods/services list is sent to the browser and DOM- manipulated
Session-based auth & multi-server scaling	HttpSession is in-memory (default Spring Security)
Currency precision/accuracy	BigDecimal used throughout instead of float/double
Media file size/bandwidth	Creation of the Supabase's partnership to harbour image and video storage securely in buckets
Data Durability	Reliability on the RLS security policies on Supabase along with SecurityContext token strategies and PostgreSQL's ACID properties

6.1 Test Cases

- Injected Database with 5 filler instances to test **CRUD** (Create, Read, Update, Delete) functionality in side the SpringBoot Application.

Tested the discover-checkout workflow – Simulating a transaction requires balance check along with customer information lookups for fund transfers and conflict resolution.

- User Detail Editing – Any user whether a customer / merchant has the ability to view, change his / her details using a password-supported way avoiding unauthenticated access and modifications
- Transaction / Appointment Simulation – Any customer can purchase goods and services using a in-website wallet tokens used to persist monetary transactions without a licensed payment gateway service.

6.2 Test Procedure

Conducted tests based on filler data – Reporting that all users were easily onboarded micmicking real-life instances

Accessed all loopholes to generate errors such as password inaccuracy, server-side error (500), client-side errors(403, 404).

Checked and serviced all loopholes considering security breaches, involuntary and incorrect SQL injections and other attacks considering multiple client requests

Held required lookups for any invalid path arrivals, authentication token bypasses, identity mismatching

6.3 Performance Outcome

```
omadb=# \dt
```

List of relations			
Schema	Name	Type	Owner
public	appointments	table	postgres
public	customers	table	postgres
public	goods	table	postgres
public	merchants	table	postgres
public	services	table	postgres
public	transactions	table	postgres

(6 rows)

Figure: Database Relations

```
omadb=# select * from merchants;
```

merchant_id	merchant_name	email	password	phone	created_at
4	Beauty Studio	beauty@shop.com	\$2a\$10\$.Lu0TUPe3Dtb5QIp0wDktOREGVcRjMaXEdYiSFKVRruKvYdVhvyLu	9876543213	2026-07-17 20:27:42.25631
5	Home Repair Experts	repair@shop.com	\$2a\$10\$6lrgCuuJaQC7Z6TeGyEBQeexOgjXyulqGcX6mTca5cxmbd4ogMEfq	9876543214	2026-07-17 20:27:42.25631
1	Smith's Electronics	john@shop.com	\$2a\$10\$z3VTr3dsG5L5DLq4YL10BuVEHT5YrWasBDaR3NWC5kFI.S4pvLLQ6	9876543210	2026-07-17 20:27:42.25631
2	Fresh Mart	fresh@shop.com	\$2a\$10\$6JCGgkd3rIAeH0nM/BxG6.dP7RBTGKlKf0aGEjjjVEKtXw7uJkM9y	9876543211	2026-07-17 20:27:42.25631
3	Elite Fitness	elite@shop.com	\$2a\$10\$IpE/PPPCXB8yBF5g20W4Q.7GqEFXrgWR89IUK6E4PoMhNCiDYhdZ.	9876543212	2026-07-17 20:27:42.25631

```
(5 rows)
```



```
omadb=# select * from customers;
```

customer_id	customer_name	email	phone	password	created_at
1	Alice Johnson	alice@gmail.com	9991111111	\$2a\$10\$2j82fwYdtZNZU.mWuxaluUv.TgbAqylTep1.3aldXAAMNnufPasi	2026-07-17 20:27:4
2	Bob Smith	bob@gmail.com	9991111112	\$2a\$10\$bKruO6ddA4RFuB6KHGthbuzTHoCNEJMcARTmdNRhWpV2UNaPyM.f0	2026-07-17 20:27:4
3	Charlie Brown	charlie@gmail.com	9991111113	\$2a\$10\$tNLKoAguz.YfFwX0BPcENeUCwkjf2B5g5IQHLeikVSL0ro6pgam2	2026-07-17 20:27:4
4	David Lee	david@gmail.com	9991111114	\$2a\$10\$dikhSvB2kkO2.uDQvh9/rOrguvglSpTXhYHD7rk2r/80a/3oUK4Xu	2026-07-17 20:27:4
5	Emma Watson	emma@gmail.com	9991111115	\$2a\$10\$K9WksZkovrYRYkKQB3P5A01bDzvuqvk7pLJ.H8/jnTk9DF5Ar60hi	2026-07-17 20:27:4
6	Abhishek Chauhan	abhishekchauhan@gmail.com	9111111110	\$2a\$10\$5rEFPZS.JNAjUu3e660mL0x46/bRLjw4/WIXZRLg6KlkzuPdRJuv2	2026-07-18 17:16:4

```
(6 rows)
```

Figure: Testcase Data for Merchants and Customers

Sales **Profile**

TRANSACTIONS **APPOINTMENTS**

1 result

DATE	PRODUCT	CATEGORY	QTY	UNIT PRICE	TOTAL	CUSTOMER
01 Jan 2026	Product	Category	1	₹0	₹0	Customer

No transactions yet.

Figure: HTML Template for Merchant Dashboard

✓

Confirmed
Subtitle

Item	Name
Reference	#0
Total Paid	₹0

Contact
Name: Merchant
Email: merchant@mail.com
Phone: -

This contact info is shown for 1:10 – reach out for status updates on your order.

Go to Discover

Figure: Checkout Page for Customer

7 My learnings

Undertaking a full-stack, multi-tenant marketplace project using Spring Boot and Thymeleaf is a significant achievement that touches upon almost every critical aspect of modern software engineering.

7.1.1 Backend Architecture & Spring Framework

- **IoC & Dependency Injection:** Deep understanding of how the Spring container manages object lifecycles (`@Service`, `@Repository`, `@Controller`).
- **Layered Architecture:** Mastery of the separation of concerns, ensuring that the presentation layer (Thymeleaf), business logic (Services), and data access (DAO/Repositories) remain decoupled.
- **Security & Authentication:** Implementing `Spring Security` to handle role-based access (RBAC). A student will learn how to secure routes so that a customer cannot access a merchant's dashboard or admin tools.

7.1.2 Database Management & Data Integrity

- **Relational Database Design:** Moving beyond basic tables to understanding **Entity Relationship Modeling**. You will learn how to manage One-to-Many relationships (e.g., one merchant to many goods) and the importance of Foreign Keys.
- **Transactional Integrity:** A crucial lesson is using `@Transactional`. You will learn why it is vital to ensure that a balance update and an order creation happen as a single atomic operation—either both succeed or both fail.
- **Query Optimization:** Learning how to write efficient SQL or JPQL to fetch data for complex views (e.g., joining tables to display an order history that lists goods, services, and dates together).

7.1.3 Frontend & Template Engine Logic

- **Server-Side Rendering (SSR):** Learning how to use Thymeleaf to dynamically inject data into HTML templates, which is a foundational skill for understanding how dynamic web content is generated on the server.
- **Data Binding:** Mastering the flow of data from the database to the browser, and the inverse—how to capture user input through forms (`th:object`, `th:field`) and map it back to Java objects.

7.1.4 Problem-Solving & "Systems Thinking"

- **Multi-tenancy Concepts:** You will learn how to build a system where the same code serves multiple users, but keeps their data strictly isolated. This is the foundation of SaaS development.
- **Business Logic Complexity:** You will learn to handle the nuance between "Goods" (inventory-based) and "Services" (time-based). Balancing these two different types of products in one system is a classic challenge in software engineering.
- **State Management:** Tracking the lifecycle of an order or appointment from "created" to "completed." You learn that a web application is essentially a "state machine."

8 Future work scope

For evaluation into a solid marketplace foundation and industry-ready platform, we can focus on these components of software development.

- **Recommendation Engine:** Implement a basic recommendation algorithm that suggests goods or services to customers based on their past purchase history or browsing habits on the "Explore" page.
- **Dynamic Pricing:** Allow merchants to set "surge pricing" or seasonal discounts for services based on demand or time of day (e.g., higher rates for peak-hour appointments).
- **Automated Conflict Resolution:** Use a simple rules engine to automatically handle common disputes, such as auto-refunding a customer if a merchant cancels an appointment within a specific window.
- **Real-time Notifications:** Integrate **WebSockets (via Spring WebSocket)** to provide real-time updates to merchants (e.g., "New Order Received!") and customers (e.g., "Your appointment is in 1 hour!").
- **In-App Messaging System:** Add a chat module between customers and merchants so they can discuss service requirements or delivery details without moving to external apps (like WhatsApp).
- **Calendar Integration:** Allow customers to export their appointments directly to Google Calendar or Outlook using an iCalendar (.ics) file generation feature.
- **Escrow Service:** Enhance the `Transactions` logic to act as an escrow. Funds from the customer's `wallet_balance` could be held in a "pending" state until the merchant marks the good as delivered or the service as completed.
- **Role-Based Access Control (RBAC) Expansion:** Move beyond simple role checks to a more granular `Spring Security` setup (e.g., `ROLE_MERCHANT_STAFF`, `ROLE_ADMIN_SUPPORT`) to allow merchants to add team members to their accounts.
- Provide merchants with visual data representations (using a library like **Chart.js** integrated with your Thymeleaf pages) to show trends in their sales, peak booking times, and customer retention rates.